Boku-nano

限られた日本語指示から
Python関数を生成する小型言語モデル



Boku-nano：限られた日本語指示からPython関数を生成するSLM

24種類の整数リスト操作を扱う、小型言語モデルをフルスクラッチ学習

① 制限された日本語指示文

整数リストxsと整数kを受け取り、
偶数だけを残し、
各要素に k を加える
solve 関数を書いてください

決められた操作・文型の範囲

→

② Boku-nano

語彙数2048のトークナイザ
1M, 5M & 15M パラメータ
トークナイザとモデルをゼロから学習
decoder Transformerによる
next token prediction

日本語指示から、python関数を生成

→

③ Python関数

```
def solve(xs, k):
    return [x + k for x in xs
            if x % 2 == 0]
```

参照インタプリタによる実行・検証

作り方と確かめ方

1

中間表現を定義

訓練データ合成のための
中間表現（意味AST）
24基本操作を最大3つ組み合せ

```
{"sequence": [{"filter": ["even"]},
              {"map": ["add_k"]}]}
```

2

合成データ作成

意味ASTから、日本語指示文と
pythonコードを生成

それぞれ基本操作に対して、20通りの
日本語表現とpythonコード合成し、
ルールベースで組み合わせる
訓練データ 約19万件

3

フルスクラッチ学習

トークナイザと
transformer をゼロから学習

15M parameter
8 layers・6 heads
1ヘッドあたり64次元

4

テスト検証・
ベンチマークによる評価

出力されたコードを
参照インタプリタで検証

84,180 / 84,550問 合格

意味ASTは24種類、3操作まで組み合せで12,720通り

基本操作

種類	操作数	例
条件に合う数を残す	10	偶数、奇数、正の値、kより大きい値
一つずつ値を変える	8	kを加える、2倍、絶対値、二乗
順序を変える	3	昇順、降順、現在の順序の反転
一部を取り出す	3	先頭k個、末尾k個、1個おき

操作の組み合わせ

操作数	数え方	意味の種類
1操作	24	24
2操作	24 × 23	552
3操作	24 × 23 × 22	12,144
合計	24 + 552 + 12,144	12,720

意味ASTの例

日本語：奇数だけを残し、各要素を2倍して降順に並べる

```
{
    "sequence": [
        {"filter": ["odd"]},
        {"map": ["mul_const", 2]},
        {"order": "descending"}
    ]
}
```

訓練・検証・テストは、意味ASTの段階で分ける

> **日本語文やコードを生成する前に、意味AST単位で分割する。**

意味ASTを分割　→　**各集合の中で**言い換え・コード変換

訓練 **約80%** ／ 検証 **約10%** ／ 通常テスト **約10%**

テスト	何を変えて確かめるか
通常	同じ基本操作の範囲で、訓練に含まれない意味ASTを評価する。
言い換え	既知の単独操作に、訓練で使わなかった日本語表現を使う。
組合せ汎化	例：別々に学んだ「偶数抽出」と「降順整列」を組み合わせる。
同じ操作の反復	同じ操作を2回・3回続けて適用する。例：「各要素にkを加える」を2回行う。
境界値	空リスト、要素1個、条件を満たす要素なし、全要素同値、負数など。

生成してから分けると、同じ問題の表記違いが訓練とテストに混入する。
同じ意味ASTから作った言い換え・等価コードは、同じ分割先にまとめる。

意味ASTから訓練データを自動生成・検証する

共通の処理の定義：意味AST

```
{"sequence": [
    {"filter": ["even"]}, {"map": ["add_k"]}
]}
```

日本語の指示文

生成の手順

Qwen3-4B-AWQで各操作の日本語表現を生成

ASTの順に結合し、テンプレートの `{operations}` をルールベースで埋める。

生成例

整数リストxsと整数kを受け取り、
偶数だけを残し、各要素にkを加える
solve関数を書いてください

検証済みのPythonコード

生成の手順

ASTの操作をPythonコードへ変換

構文を確認し、実行結果を
参照インタプリタの結果と検証

生成例

```
def solve(xs, k):
        return [x + k for x in xs
                        if x % 2 == 0]
```

Qwen3-4B-AWQで日本語表現を生成し、組み合わせで指示文作成

表現辞書の作成と、**承認済み表現の組み合わせ**の二段階で指示を作る。

① 表現辞書を作る：Qwen3-4B-AWQ ＋ 人

Qwen3-4B-AWQに、各基本操作の表現を約20種類作成

保存済みプロンプト（要約）

> 意味ASTと厳守事項に従い、指定件数の表現を作る。
> 操作・比較境界・入力順序を変えず、別の操作を加えない。
> 終止形と接続形を同じ意味で作り、指定のJSONだけを返す。

例：「偶数を残す」の生成候補

「偶数だけを残す」／「偶数だけを残し」
「偶数を選び出す」／「偶数を選び出して」

人が意味を確認し、承認した表現だけを辞書に登録

② 指示を作る：ルールベース

意味ASTに沿って、承認済みの表現を組み合わせる

例：偶数を残す → 各要素にkを加える

選ぶ表現

「偶数の要素を取り出す」
「各要素にkを加える」

つなぐルール

操作1し、操作2する

生成する日本語指示

「偶数の要素を取り出し、各要素にkを加える」

意味ASTから、ルールで等価なPythonコード生成

同じ意味ASTでも、異なる書き方のコードを生成

内包表記でまとめて選ぶ

```python
def solve(xs, k):
    # 条件に合う要素のリストを返す
    return [
        value
        for value in xs
        if value % 2 == 0]
```

forループで順に選ぶ

```python
def solve(xs, k):
    result = []
    for value in xs:
        if value % 2 == 0:
            result = result + [value]
    return result
```

共通の入力： `[3, 2, 4, 2]` → どちらの出力も `[2, 4, 2]`

変える箇所	変換例
処理の構造	内包表記 ↔ forループ ／ 直接return ↔ 一時変数を使う複数行
名前・補足	変数名の変更 ／ コメント・型注釈の有無
条件・並べ替え	独立した条件の順序変更 ／ `reverse=True` ↔ 昇順ソート後の逆順（整数）

生成コードは、完全重複がなく、意味ASTの参照実装との結果一致を確認

学習に使った合成データは192,900件

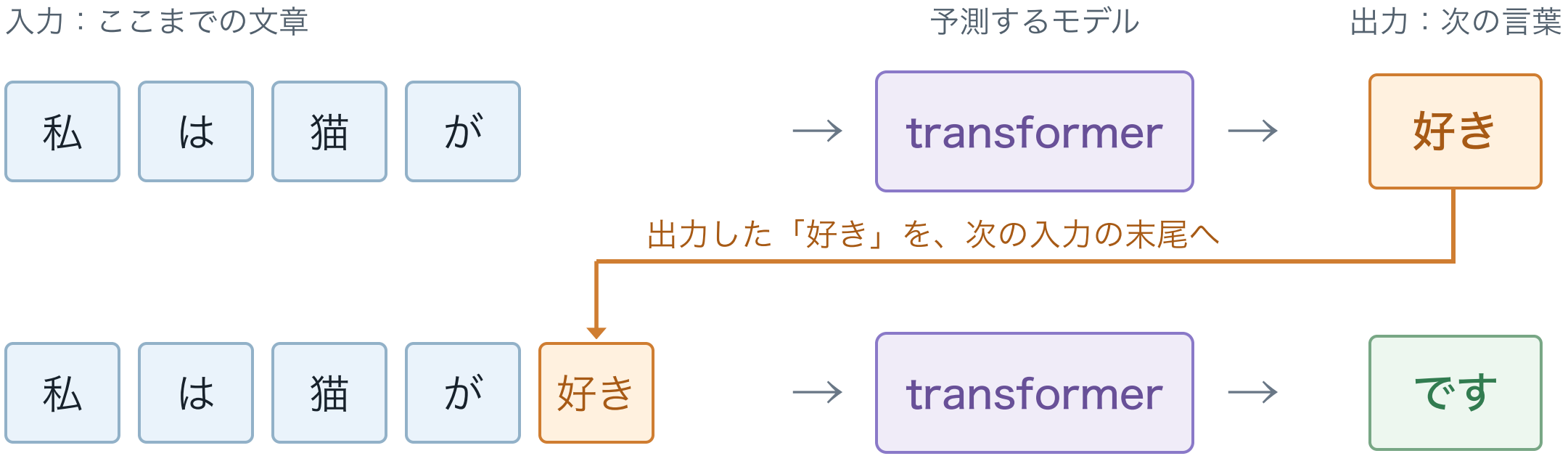
1件は「日本語の指示」と「正解コード」の組。

処理の長さ	訓練データの件数
1操作	460
2操作	8,680
3操作	183,760
合計	192,900

約95%が3操作の合成データ。 単独操作の合成データは少ない。

自己回帰：出力を次の入力に加えて繰り返す

decoder型transformerは、直前までの言葉から次の言葉を予測する。



> **学習では**、文章の途中までを見せ、実際の次の言葉を当てるように学ぶ。

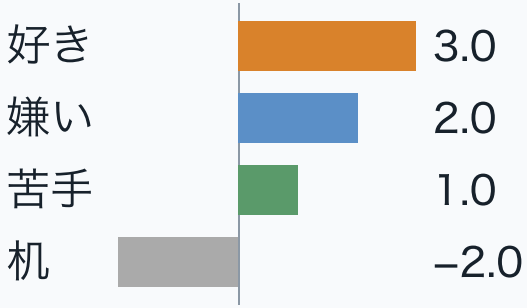
次の言葉の候補に確率が割り当てられる

「私は猫が…」の続きも、候補の点数を確率に直し、その中から1つ選ぶ。

私 は 猫 が ？ → decoder transformer → 次の候補すべてに点数をつける

① ロジット＝候補の点数

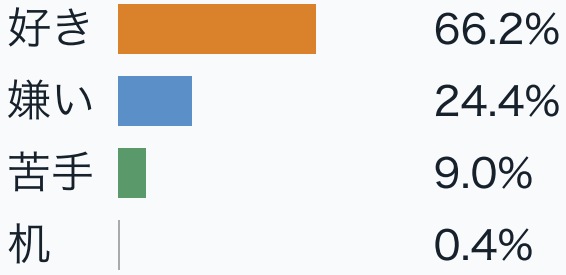
ロジットは確率ではなく、負にもなる



softmax →

② 合計1となる確率

softmaxで変換。



選ぶ →

③ 最も確率が高い候補を選ぶ

今回のBoku評価で使う選び方。

好き　66.2%

↓ 入力の末尾に追加

私は猫が**好き**

新たな入力で、次のトークンを予測する。

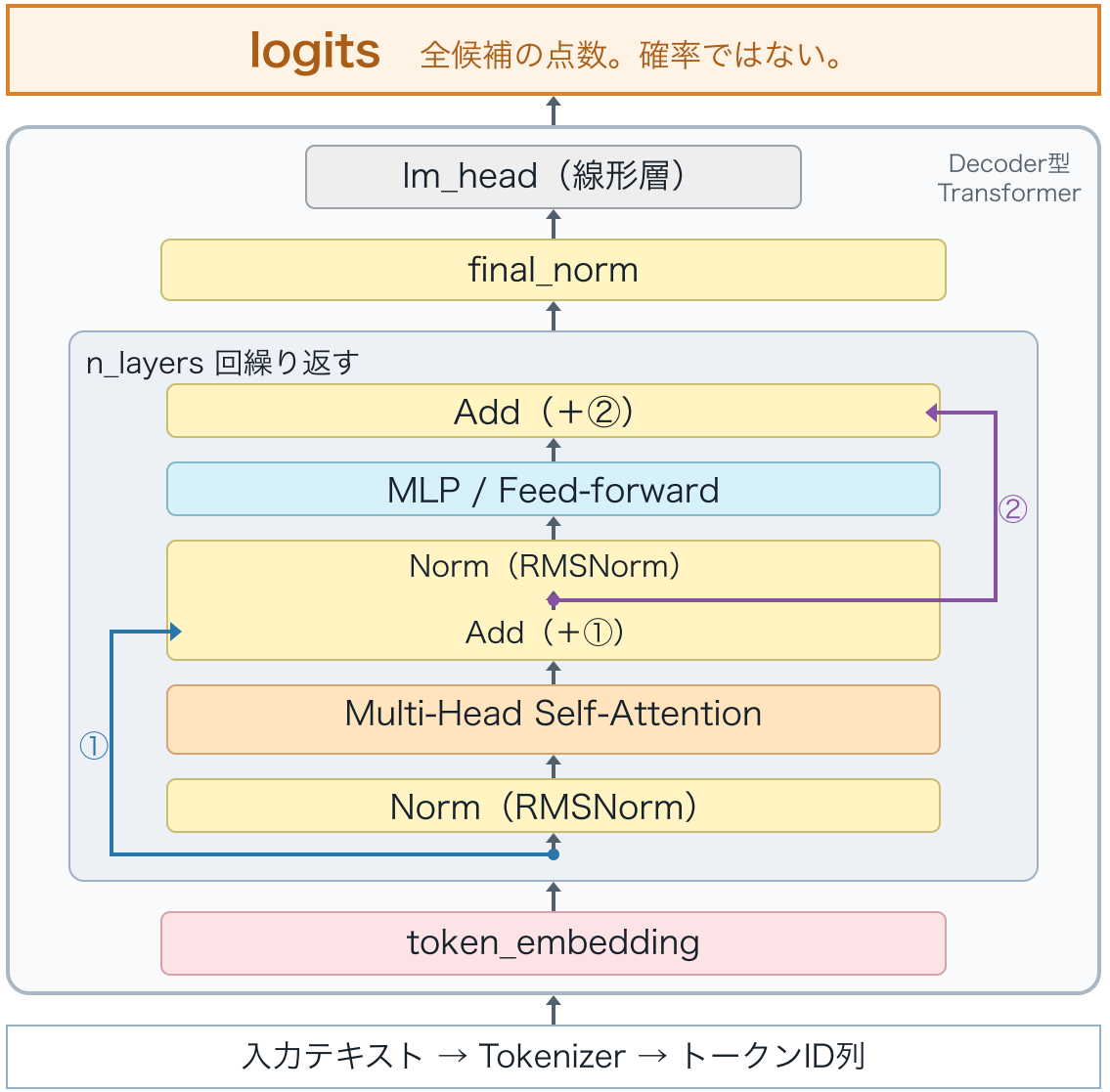
> ロジットはsoftmaxで、合計1となるそれぞれの確率に変換される。

① ロジットはDecoder Transformerモデルの出力

```
# トークンIDを埋め込みへ変換
h = self.token_embedding(input_ids)
for block in self.blocks:
    # ① 正規化前のhを保存し、Attention後に足す
    residual = h
    h = block.attention_norm(h)
    h = block.attention(h)
    h = residual + h
    # ② 正規化前のhを保存し、MLP後に足す
    residual = h
    h = block.ffn_norm(h)
    h = block.feed_forward(h)
    h = residual + h
# 最後の正規化と、語彙ごとの点数への変換
h = self.final_norm(h)
logits = self.lm_head(h).float()  # ここがロジット
# 推論時はロジットを返す
if labels is None:
    return CausalLMOutput(logits=logits, loss=None)
```

`lm_head` の出力が、全候補の点数（ロジット）。

boku_nano.pyのforwardとブロック内部を展開して記載。
hidden_statesをhと表記し、残差加算を分けて示した。
図の①・②は、同じ番号のresidualをそのまま足す経路。



② softmaxは、点数を「合計1の確率」に変換する

```
# 入力の最後の位置にある、全候補の点数
# このページでは temperature=1.0 とする
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補を点数の高い順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# 点数を、合計1の確率分布に変換する
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

evaluate_boku_nano_comparison.py：209–211行。
コメントを追加し、改行を調整。

指数に変換し、全候補の合計で割る

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

z_i：候補 i の点数、p_i：その選択確率

V：候補数、$\sum$：全候補の足し算、$e \approx 2.718$

候補	点数 z_i	e^{z_i}	確率 p_i
A	2	7.389	66.5%
B	1	2.718	24.5%
C	0	1.000	9.0%

$$p_A = \frac{7.389}{7.389 + 2.718 + 1} \approx 0.665$$

③ 選んだトークンを入力に加え、予測を繰り返す

```
# 日本語指示を含む入力ID列をコピーする
current = prompt_ids.copy()
generated: list[int] = []

# 勾配を計算せず、次のトークンを順に生成
with torch.inference_mode():
    for _ in range(min(
        max_new_tokens,
        model.config.context_length - len(current)
    )):
        values = torch.tensor(
            [current], dtype=torch.long)
        # 最も確率が高いトークンを選ぶ
        token_id = int(torch.argmax(
            model(values).logits[0, -1]).item())
        # 終了トークンなら生成を止める
        if token_id == eos_token_id:
            break
        # 選んだIDを出力と次回の入力へ追加
        generated.append(token_id)
        current.append(token_id)
```

Python推論から抜粋。コメントを追加し、改行を調整。
export_boku_nano_onnx.py：232–241行

`logits[0, -1]` は先頭の入力系列の **最後の位置** にある、全候補の点数。

自己回帰の中心は `current.append`

1. **ここまでの入力**から次の候補を予測する。
2. 候補を1つ選び、**入力末尾に追加**する。
3. 伸びた入力を使って、**同じモデル**でもう一度予測する。

各ステップで、最も確率が高いトークンを選ぶ

選んだトークンを入力に加え、次の予測でも **最も確率が高いトークン** を選ぶ。終了トークンか長さの上限まで繰り返す。

softmaxで確率に変換しても、候補の順位は変わらない。

そのため、コードでは **ロジットが最大の位置を求める** `argmax` で、同じトークンを選べる。

④ 温度は、候補の確率の偏りを調整する

```
# 全候補のロジットを温度Tで割る
logits = (
    model(input_ids).logits[:, -1, :]
    / temperature
)

# 候補のIDを保持して点数順に並べる
sorted_logits, sorted_indices = torch.sort(
    logits, descending=True, dim=-1
)

# 調整後の点数をsoftmaxで確率にする
sorted_probabilities = torch.softmax(
    sorted_logits, dim=-1
)
```

evaluate_boku_nano_comparison.py：209–211行。
確率に従って抽選する場合の実装から抜粋。
今回のBoku評価では、最も確率が高いトークンを選ぶ。
参考：Aman AI：Temperature

抽選する場合の分布を温度で調整する

$$p_i(T) = \frac{e^{z_i/T}}{\sum_{j=1}^{V} e^{z_j/T}}, \quad T > 0$$

T：生成時に指定する温度。$T = 1$ が基準。
学習済みの重みは変えず、ロジットを T で割る。

AとBの点数が2と1なら、差は1。割った後の差は $1/T$ になる。

温度 T	Aの点数 $2/T$	Bの点数 $1/T$	差
0.5	4	2	2
1	2	1	1
2	1	0.5	0.5

$0 < T < 1$：点数差が広がり、高確率の候補に集中する。

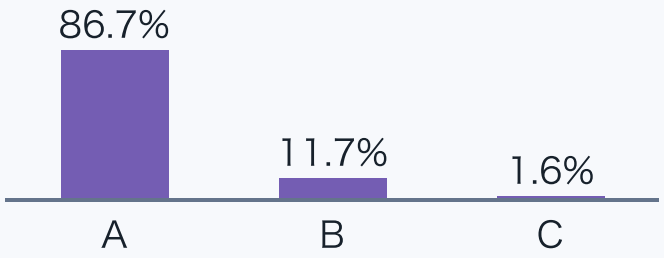
$T > 1$：点数差が縮まり、分布が平らになる。

⑤ 温度を上げると、低い確率の候補も選ばれやすくなる

ここでは3候補 A=2、B=1、C=0 の点数を固定し、温度だけを変えて比べる。

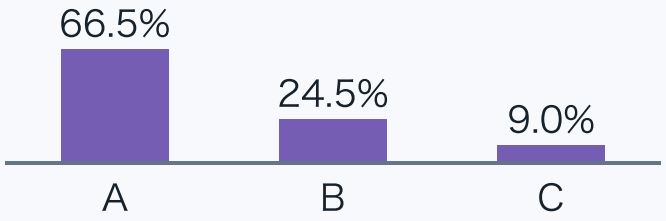
T = 0.5：Aに集中

割った後の点数：[4, 2, 0]



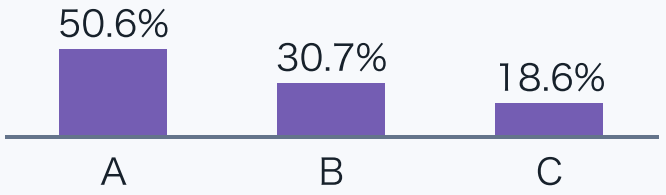
T = 1：基準

割った後の点数：[2, 1, 0]



T = 2：B・Cも選びやすい

割った後の点数：[1, 0.5, 0]



AはBの何倍選ばれやすいか（棒の高さは共通尺度：0〜100%）

$$\frac{p_A(T)}{p_B(T)} = e^{(z_A - z_B)/T} = e^{1/T}$$

$T = 0.5$ なら約 7.39倍、$T = 1$ なら約 2.72倍、$T = 2$ なら約 1.65倍。

確率に従って抽選する場合は、温度を上げると出力のばらつきが増える。

最も確率が高い候補を選ぶ場合は、正の温度を変えても選ぶ候補は同じ。

参考：Aman AI：Temperature。温度は正解率の向上を保証しない。

モデル構成とトークナイザー

項目	1M	5M	15M
総パラメータ数	1,016,704	5,065,472	15,735,168
隠れ次元 `d_model`	128	256	384
層数 `n_layers`	3	5	8
Attentionのhead数	4	4	6
1 headの次元	32	64	64
FFNの中間次元 `d_ff`	256	704	1,024
この後に示す学習曲線	3 epoch	1 epoch	1 epoch

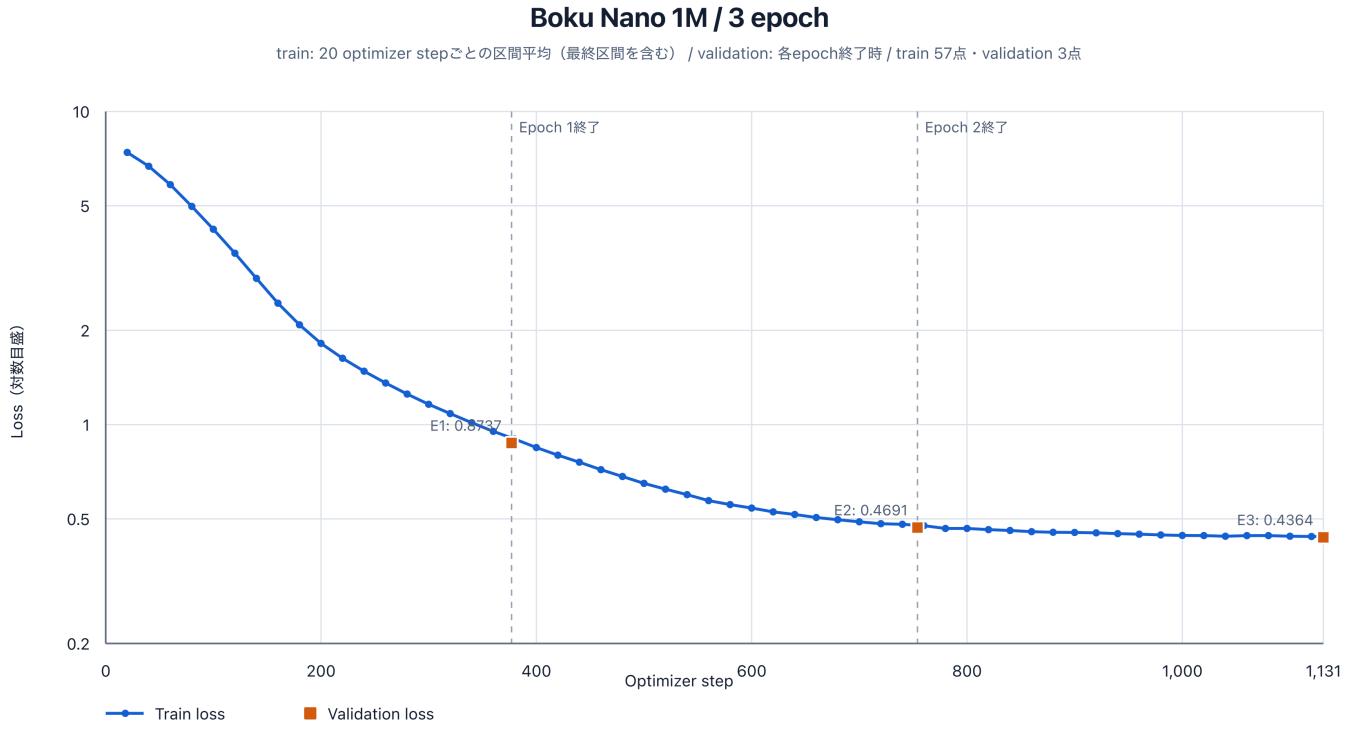
トークナイザーは共通

BPE ／ 語彙数 2,048

最小頻度 `min_frequency = 5` ／ 最大piece長 `max_token_length = 24`

1M：3 epochまでの損失の推移

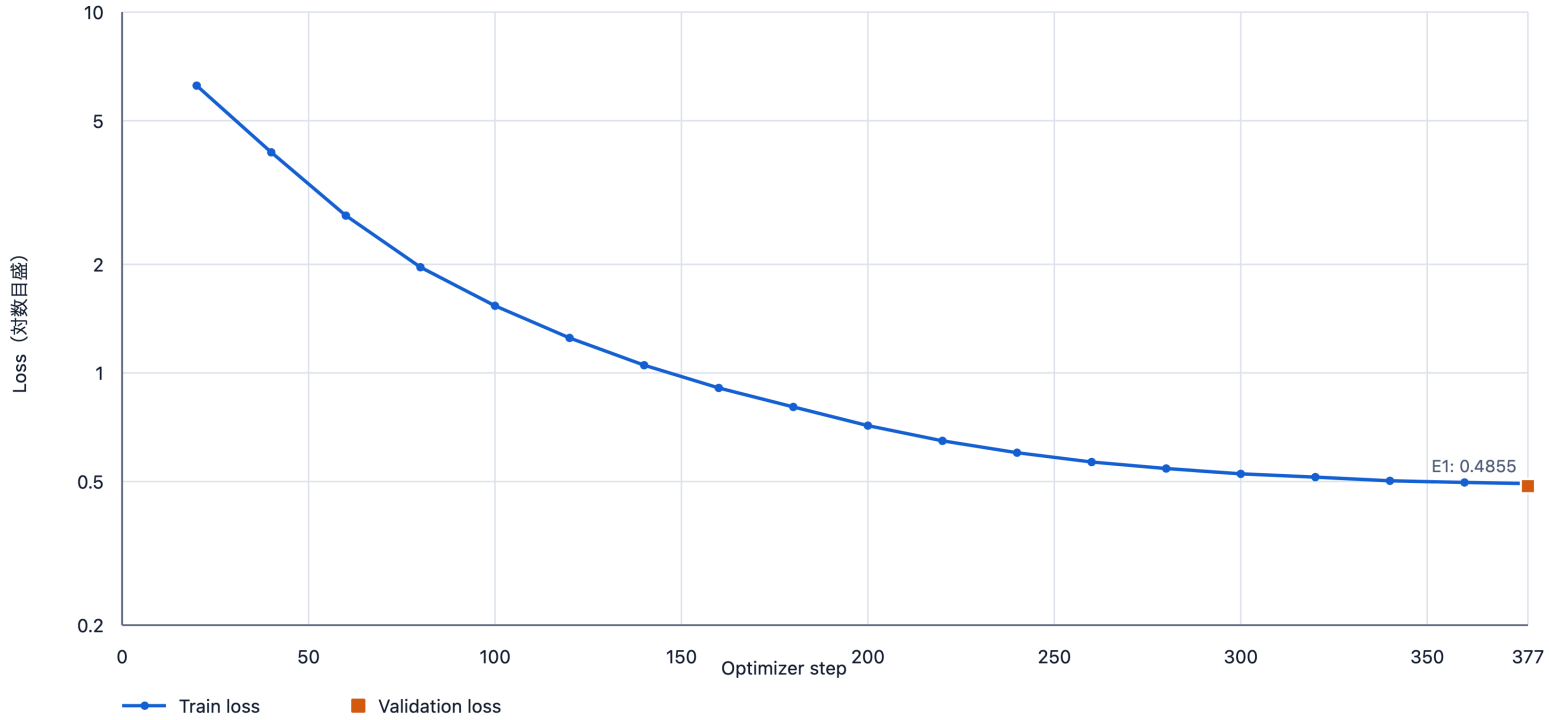
3 epochの学習。訓練損失は20 stepごと、検証損失は各epoch終了時の実測値。



val loss：1 epoch 0.8737 ／ 2 epoch 0.4691 ／ 3 epoch 0.4364

5M：損失の推移

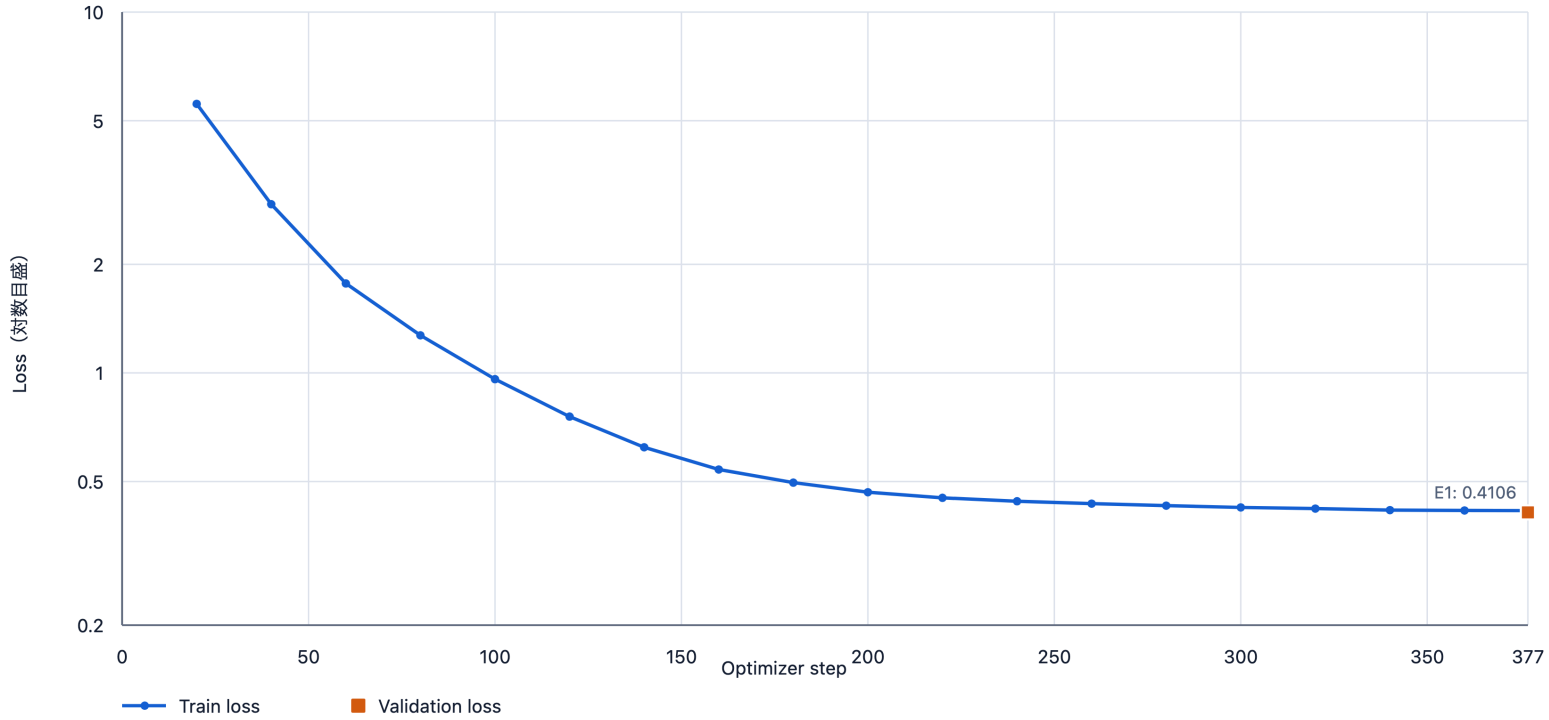
1 epochの学習。訓練損失と、検証データで測った損失を示す。



学習終了時の val loss：0.4855

15M：損失の推移

1 epochの学習。訓練損失と、検証データで測った損失を示す。



学習終了時の val loss：0.4106

モデル規模・学習回数による違い：五つのテスト別結果

各モデルの列は **合格数・合格率**。生成コードが各問の検査入力をすべて満たせば合格。

テスト	1M・1 epoch	1M・3 epoch	5M・1 epoch	15M・1 epoch
通常（24,040問）	446問（1.86%）	23,950問（99.63%）	23,261問（96.76%）	**24,003問（99.85%）**
組合せ汎化（13,400問）	103問（0.77%）	**13,311問（99.34%）**	12,657問（94.46%）	13,188問（98.42%）
日本語の言い換え（30問）	2問（6.67%）	**15問（50.00%）**	8問（26.67%）	**15問（50.00%）**
同じ操作の反復（23,040問）	508問（2.20%）	22,945問（99.59%）	22,370問（97.09%）	**22,971問（99.70%）**
境界値（24,040問）	408問（1.70%）	23,950問（99.63%）	23,261問（96.76%）	**24,003問（99.85%）**

1Mは3 epochで大きく改善。一方、言い換えは1M・3 epochと15M・1 epochでも50%。

共通の既存BPE設定・単一seed。通常と境界値は同じ指示に異なる検査入力を使用。
ここでの固定5テストと、次に示す872問の組み合わせ汎化ベンチマークは別の評価。

ベンチマークの役割と、自分の研究での作り方

モデルの能力と限界を観測し、次に取り組む研究課題を明確にする。
成功例だけで判断せず、共通の基準で能力の向上を定量的に確かめる。

設計・評価の手順	自分の研究で考えること
① 測りたい能力を決める	何ができるようになったかを確かめたいのか
② 課題と正解基準を作る	何を変え、何を固定し、どう正誤を判定するか
③ 共通条件で測り、分析する	成功・失敗の条件を調べ、比較条件と作成手順を記録する
④ 次の課題につなげる	残る誤りを調べ、高得点で差が見えなくなれば評価を見直す

一例：組み合わせ汎化を測る

学習済みの操作を、新しい組み合わせでも実行できるか。
今回は既知の日本語表現を使って構成を変え、生成コードの実行結果で正誤を判定する。

データセットの構成：9種類・872問

操作数	問題の種類	問数	構成
1	複数操作で学んだ表現を1操作で使う	96	24操作 × 4表現
2	初めて組み合わせる2操作	100	5ペア × 両順序 × 10表現
2	同じ操作を2回	96	24操作 × 4表現
2	3操作で学んだ順序を2操作で使う	88	88種類の操作列
3	初めてのペアを含む3操作	100	100種類の操作列
3	同じ操作を3回	96	24操作 × 4表現
4	同じ操作を4回	96	24操作 × 4表現
4	異なる4操作を組み合わせる	100	100種類の操作列
4	2操作の組を2回繰り返す	100	A・B・A・Bの100種類

各問題でコードを1つ生成。**179通りの共通テスト入力ですべて合格した問題の割合**を正答率とする。

Astraの評価設定：思考low・ツール使用を禁止

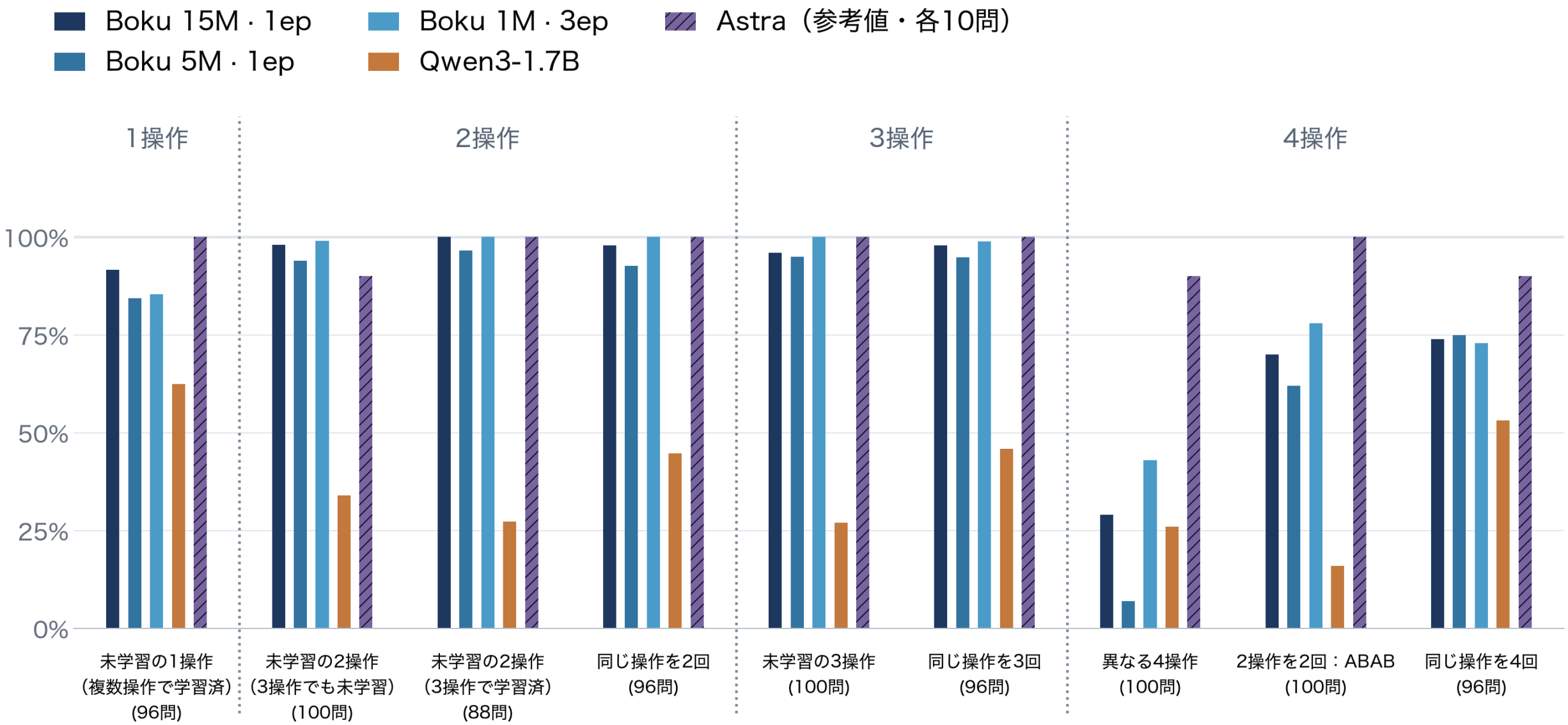
項目	設定
モデル・推論設定	`gpt-6-astra` ／ reasoning effort：low
評価対象	872問から固定seedで抽出した各カテゴリ10問、計90問
会話・回答	1問ごとに独立チャット。最初の回答1回を採点。修正・再生成なし
ツール	使用禁止を指示。**90問すべての実行記録で使用0件を確認**

各問題の冒頭に与えた指示：

> ツールは一切使用せず、このメッセージだけから回答してください。ファイル閲覧、検索、コード実行、他のエージェントへの依頼は行わないでください。回答は一度だけ、指定されたコードのみを出力してください。

ツールの技術的な無効化ではなく、**指示による禁止と記録確認**。採点は回答保存後に行い、結果をAstraへ返していない。

問題カテゴリ別のモデル比較



本評価：カテゴリ欄の問題数／Astra参考値：各10問・計90問（独立チャット・low・ツール使用なし）。評価集合・実行条件が異なるため参考比較。

高得点の課題と、残る課題を分けて読む

問題の種類	Boku 15M・1ep	Boku 5M・1ep	Boku 1M・3ep	Qwen3-1.7B	Astra・参考値
初めて組み合わせる2操作	98 / 100	94 / 100	99 / 100	34 / 100	9 / 10
初めてのペアを含む3操作	96 / 100	95 / 100	100 / 100	27 / 100	10 / 10
異なる4操作を組み合わせる	29 / 100	7 / 100	43 / 100	26 / 100	9 / 10

Astraだけ各カテゴリ10問。**分母と対象問題が異なるため、参考比較。**

- **今回の2・3操作の課題では、小規模モデルにも有望な結果がある。**
- Bokuの4操作では誤りが多く残る。どの操作を落とすか、順序を変えるかを分析する。
- 1Mは3epoch、5M・15Mは1epoch。生成条件も異なり、規模だけの優劣とは解釈しない。

今回の結果の意義と、次の研究課題

今回のベンチマークでは、小規模モデルの組み合わせ汎化について有望な結果が得られた。

今後は比較条件を揃え、より大きなモデルとの比較を追加するとともに、**評価設計と作成手順を明文化する。**

- 比較条件：訓練回数、プロンプト、生成上限などを記録し、揃えられる条件を揃える。
- 評価設計：何を既知・未知とするか、どの誤りを検出したいかを定める。
- 作成手順：問題の選定、訓練との照合、採点方法、再現手順を公開可能な形に整理する。

デモ：学習済みモデルをブラウザで試す

ベンチマークでは、2・3操作で有望な結果が得られ、4操作では課題が残った。

ここからは、自由な日本語を文型に整え、コードが生成されるまでを実演する。

ONNX形式で保存したモデルを、ONNX Runtimeでブラウザ内実行。Python環境・推論サーバーは不要。

デモの流れ	役割
① 自由な日本語を入力	作りたい処理を伝える
② Qwen3-0.6Bで文型を整える	自由文を決められた文型（CNL）に変換。コードは書かない
③ JavaScriptで指示を検査	24操作・最大3操作・定型文法に合うか確認
④ Boku-nanoでコードを生成	選択した学習済みモデルがPython関数を出力

ブラウザデモを開く

Appendix：追加解析と補足資料

本編の評価を補足し、日本語表現への弱さとモデル内部の挙動を調べる。

内容	確かめること
A. 日本語表現への頑健性	言い換えや指示の冒頭を変えると、生成結果はどう変わるか
B. Attentionの可視化と介入	どこを参照し、指示の情報が生成にどう寄与するか
C. 追加解析のまとめと限界	結果から言えることと、追加検証が必要なこと
D. 全体図	データ生成・学習・評価・デモのつながり

全体が高精度でも、言い換えには弱い

1M・3 epochと15M・1 epochの比較。各セルは合格数（合格率）。

テスト（評価件数）	1M・3 epoch	15M・1 epoch
通常（24,040問）	23,950（99.63%）	24,003（99.85%）
組合せ（13,400問）	13,311（99.34%）	13,188（98.42%）
日本語の言い換え（30問）	**15（50.00%）**	**15（50.00%）**
同じ操作の反復（23,040問）	22,945（99.59%）	22,971（99.70%）
境界値（24,040問）	23,950（99.63%）	24,003（99.85%）

どちらも全体では約99.6%だが、未学習の言い換えは50%にとどまる。

先行研究：日本語の書き方で正解率が変わる

Gan and Mori（2023）は、意味や指示が近い5種類のプロンプトを比較した。

プロンプトは、モデルに渡す指示文や質問文のこと。

日本語の課題	モデル	正解率の範囲
2文の意味関係を分類するJNLI	GPT-4	25.44〜49.21%

意味が近くても、書き方によって性能が大きく変わる場合がある。

出典：Gan and Mori, PACLIC 2023, Table 3

先行研究：言い換えで回答の傾向も変わる

Takayama et al.（2026）は、日本語を含む「はい／いいえ」の質問を評価した。

表現の変え方	日本語での観察
同意を求める「〜ですよね？」	道徳判断で「はい」が増える傾向
反対語への置き換え	答えの反転を考慮しても、一貫性が低下

同じ判断を求めても、表現によって答えが動く。

モデルの評価には、複数の言い回しを含める必要がある。

出典：Takayama et al., LREC 2026, Table 1

今回は、指示の冒頭だけを変えて比較する

条件	モデルに渡す指示
変更前	整数リストxsから偶数だけを残すsolve関数を書いてください。
変更後	整数リストxsに対して、偶数だけを残すsolve関数を書いてください。

後半の操作の指示をそろえ、冒頭だけが異なる**14操作**を比較。

同じモデルを使い、追加学習せず、同じ入力でコードを検査する。

5Mモデルでは、12件の合格が1件に減った

5M・1 epoch。比較した14操作の合格数。

指示の冒頭	合格数を表す図	合格数・合格率
整数リストxsから	■■■■■■■■■■■■□□	12 / 14・85.7%
整数リストxsに対して、	■□□□□□□□□□□□□□	1 / 14・7.1%

■は合格1件、□は不合格1件。左から合格数を並べた図。

1M・3 epochでも、同じ14操作で 12 / 14 → 9 / 14 に低下した。

冒頭を変えただけで、ほとんど正しく生成できなくなった。

失敗例：偶数を残す指示を別の処理にした

変更後の指示：**整数リストxsに対して、偶数だけを残すsolve関数を書いてください。**

	指示どおり	5M・1 epoch	1M・3 epoch
処理	偶数だけを残す	順序を反転し、k以上を残す	正の数だけを残す
同じ検査入力	`xs = [0], k = 1`	`xs = [0], k = 1`	`xs = [0], k = 1`
出力	`[0]`	`[]`	`[]`

両モデルとも構文は正しいが、**指示された処理の意味が変わった。**

表現への強さは、モデルによって違う

冒頭だけを変えた14操作の合格数。学習回数も併記する。

モデル	「xsから」	「xsに対して、」
1M・3 epoch	12 / 14	9 / 14
5M・1 epoch	**12 / 14**	**1 / 14**
15M・1 epoch	12 / 14	12 / 14

1M・3 epochでは3件減り、15M・1 epochでは合格数を保った。
変化の大きさは、モデルと学習回数によって異なる。

学習データの文型との結び付きが考えられる

訓練データで「整数リストxsに対して、」と始まる指示は84件。

その文型で学習した処理	件数
1操作	0
2操作	1
3操作	83

この文型を、複数の操作と結び付けて覚えた可能性がある。

原因の確定には、合成データを変えた追加比較が必要。

モデルが情報を読み取る仕組み

アテンションは、次のコードを作るときに、どこから情報を読むかを決める仕組み。

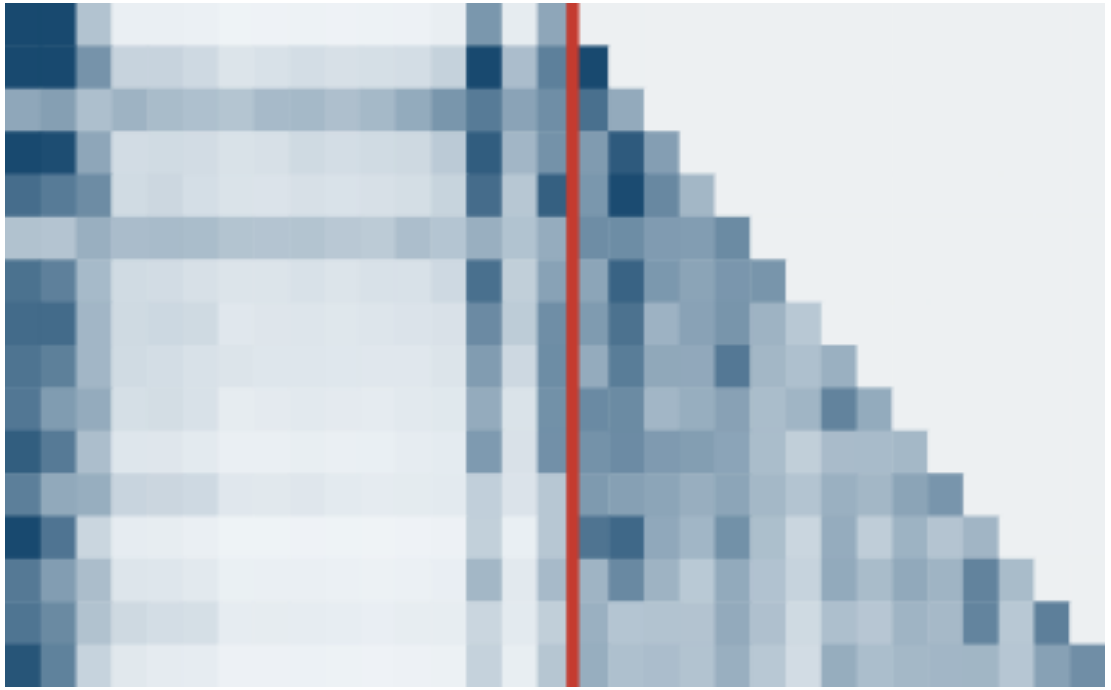
参照できる情報	内容
日本語の指示	「偶数」「残す」など、求める処理
すでに生成したコード	変数名、構文、処理の続き
開始の印など	入力の始まりや区切り

参照割合だけでは、その情報が必要だったかは確定できない。

情報の受け渡しを止めて、生成結果が変わるかを調べる。

アテンションの図の読み方

入力の指示・区切り　　生成済みコード



生成
ステップ
↓

薄い：弱い

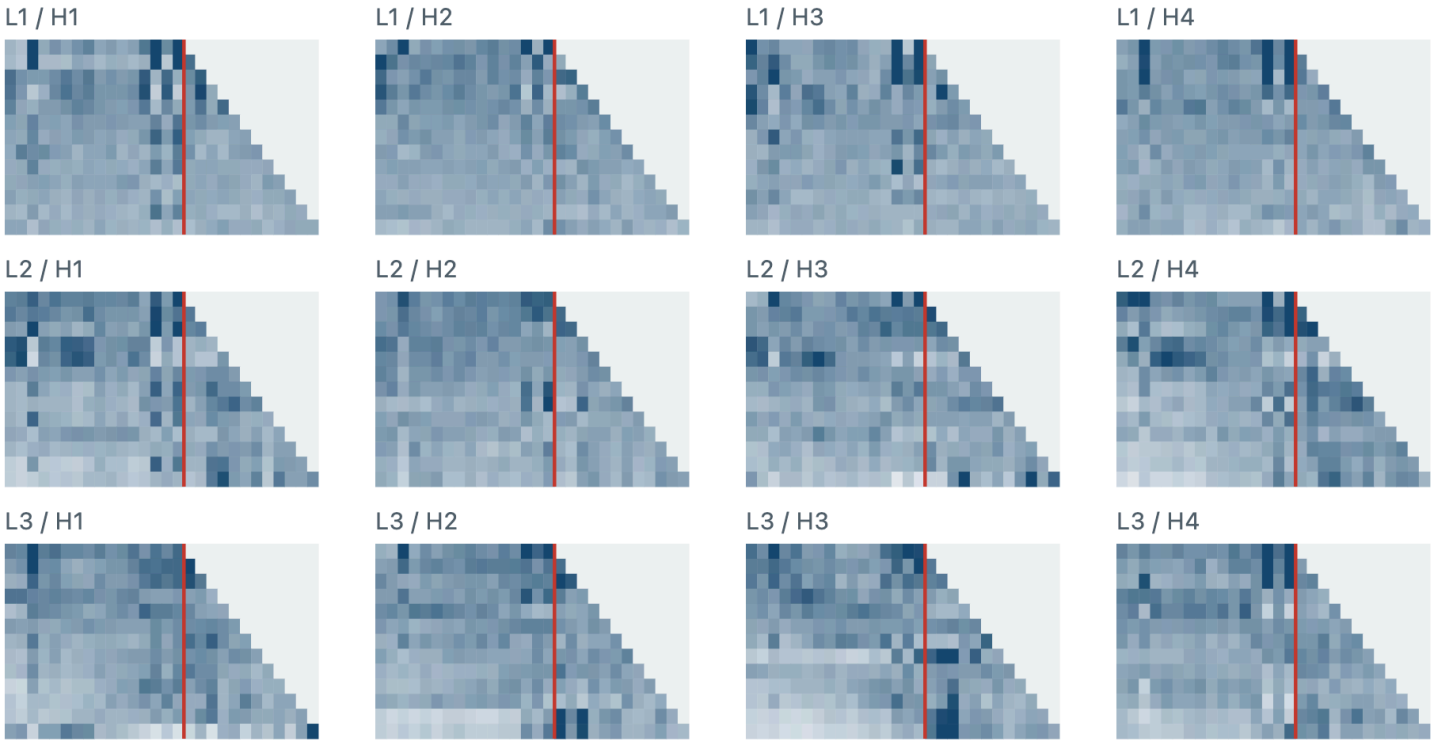
濃い：強い

横：参照するトークン位置 →

濃い位置ほど強く参照。15M・1エポックのL8・H5の例

1M・1 epoch：attention の可視化

同じ指示：**各要素を絶対値にして、降順に並べる**

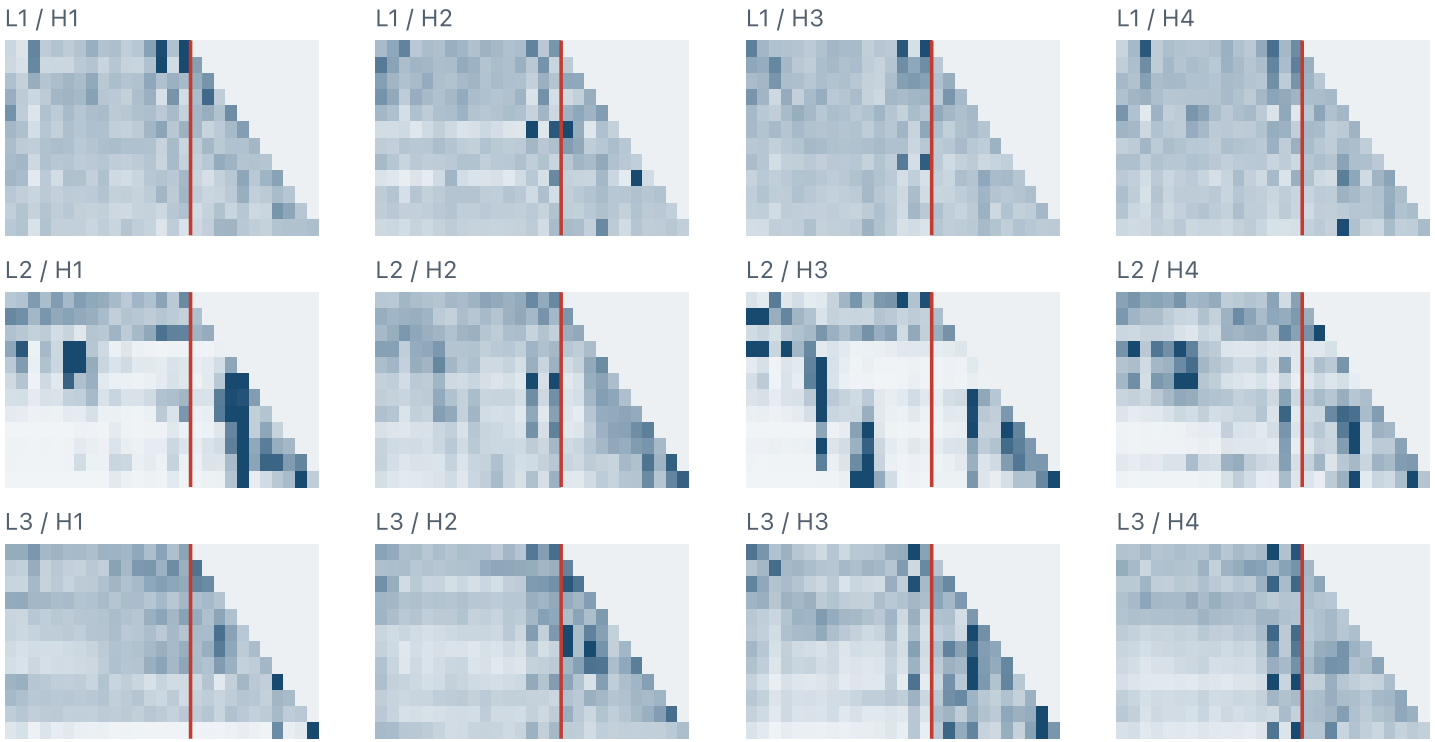


生成結果：未定義の result を参照し、この指示では実行できなかった。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

1M・3 epoch：attention の可視化

同じ指示：**各要素を絶対値にして、降順に並べる**

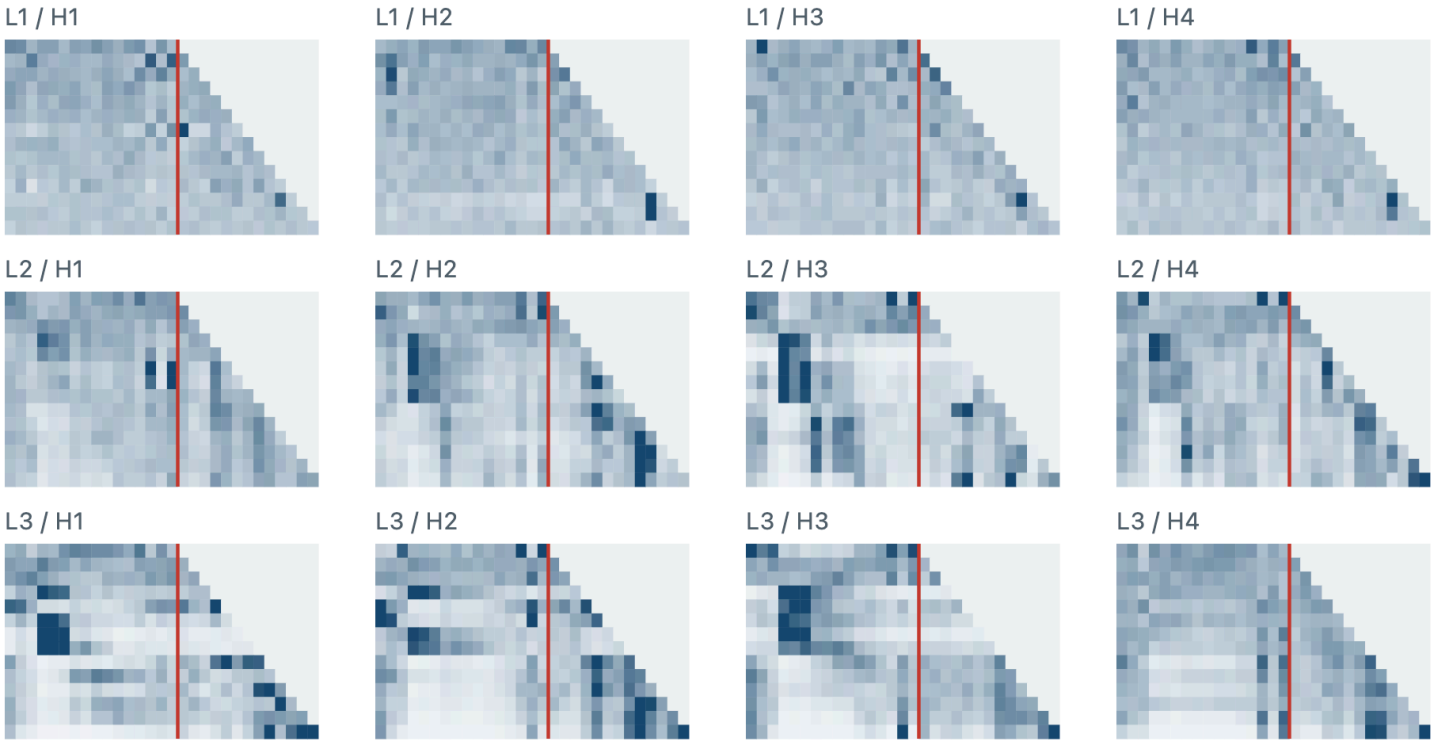


生成結果：未定義の output を参照し、この指示では実行できなかった。

3層・4 head、12生成step。色の濃さはモデル間で直接比較しない。

5M：attention の可視化（L1〜L3）

同じ指示：**各要素を絶対値にして、降順に並べる**

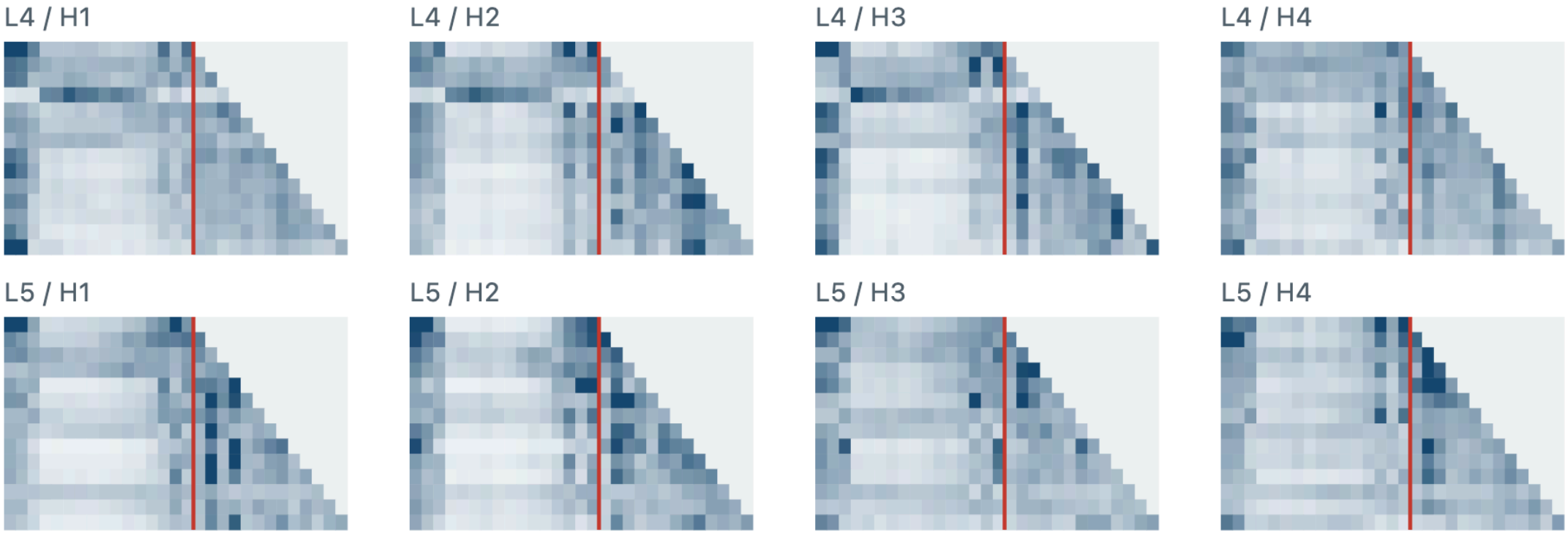


生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

5M：attention の可視化（L4〜L5）

同じ指示：**各要素を絶対値にして、降順に並べる**

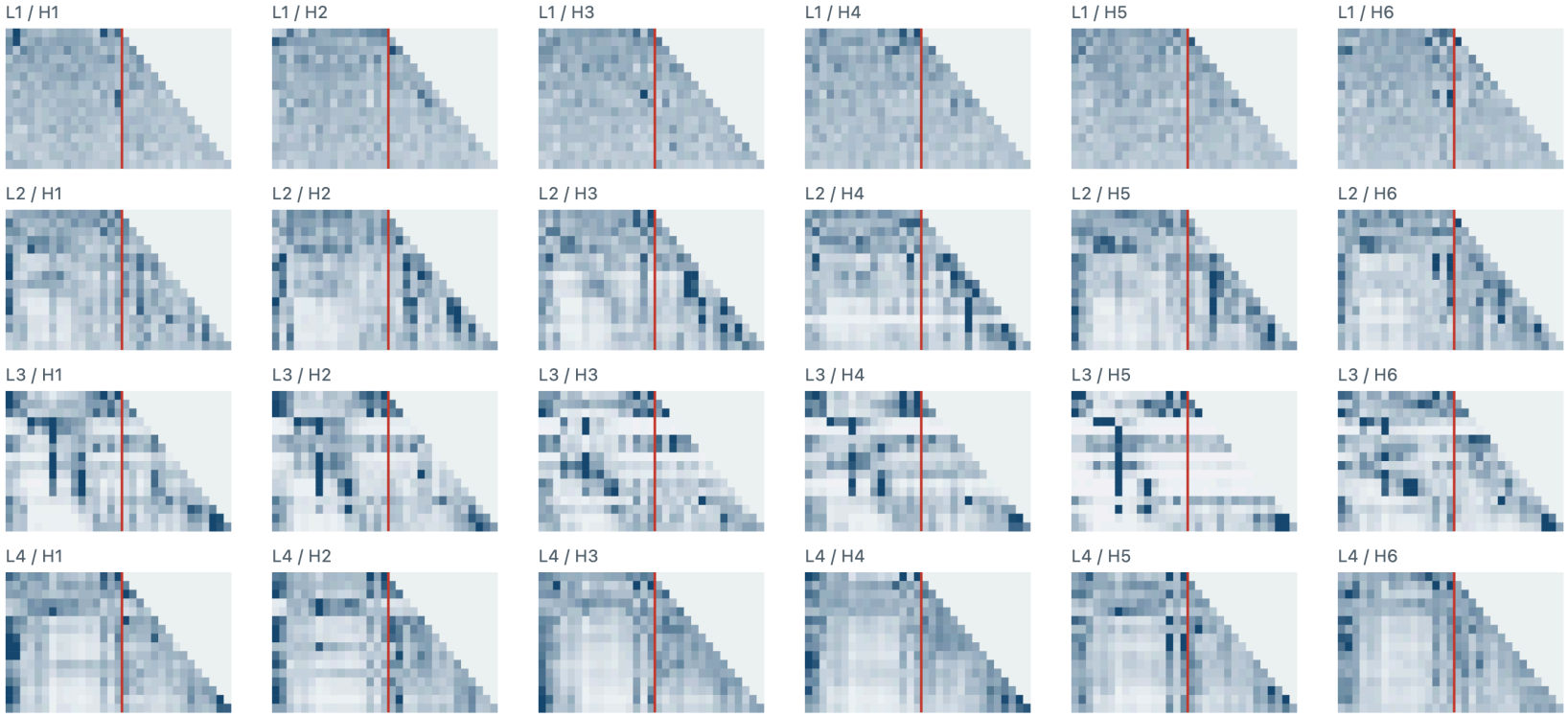


生成結果：絶対値の処理は生成したが、k加算が入り、降順の処理が抜けた。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M：attention の可視化（L1〜L4）

同じ指示：**各要素を絶対値にして、降順に並べる**

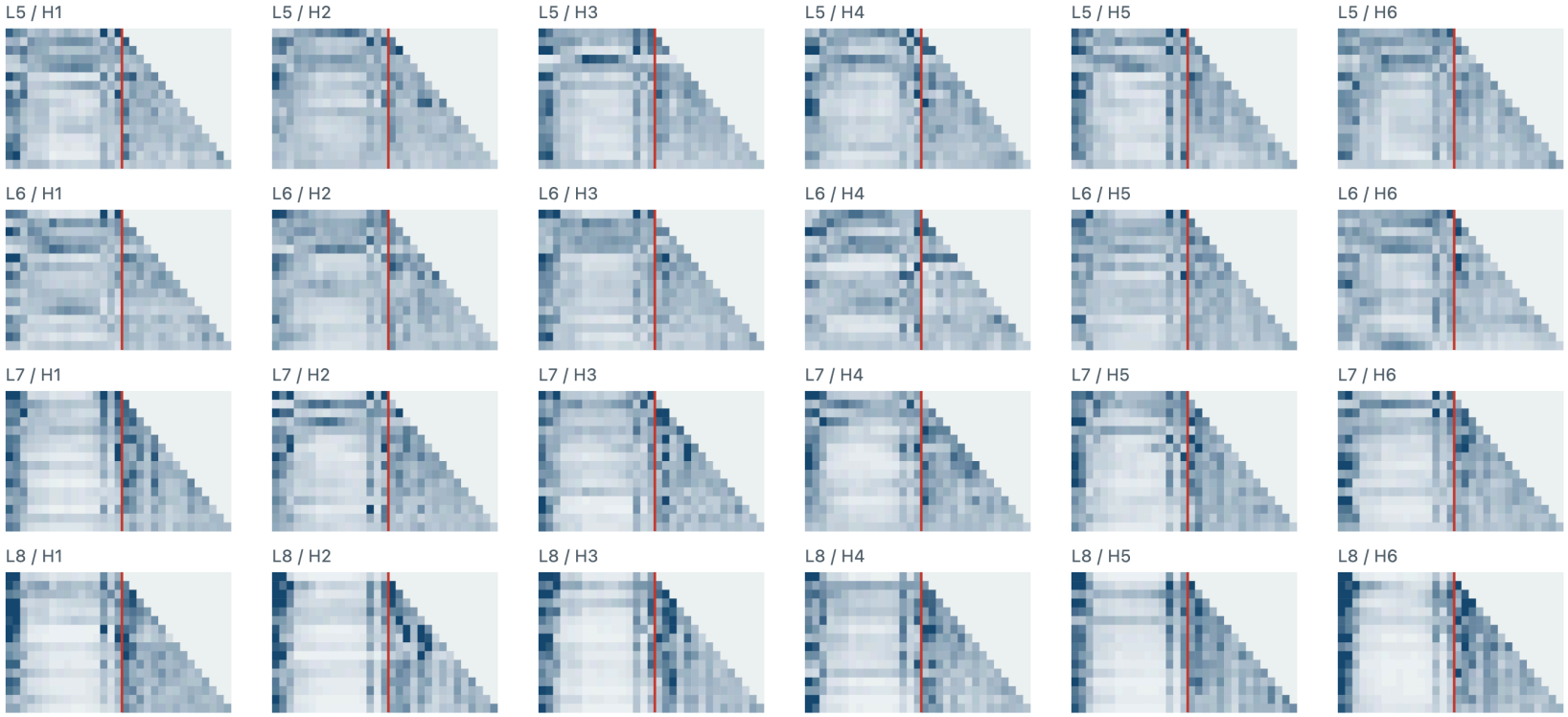


生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

15M：attention の可視化（L5〜L8）

同じ指示：**各要素を絶対値にして、降順に並べる**



生成結果：絶対値を取って降順に並べるコードを生成した。

色は各モデル内の相対的な強さ。モデル間で濃さを直接比較しない。

指示の情報を止めると合格数が落ちる

学習時に準拠した文型で、24操作を診断。

モデル	通常の生成	指示の情報を止めた生成
1M・1 epoch	3 / 24	0 / 24
1M・3 epoch	19 / 24	0 / 24
5M・1 epoch	22 / 24	0 / 24
15M・1 epoch	22 / 24	2 / 24

指示から受け取る情報が、操作の選択に寄与している。

24操作の内部診断。冒頭変更の14操作、固定5テストとは別。

評価では、表現を変えた場合も確かめる

調べること	今回、分かったこと
決めた文型での動作	1M・3 epochも高い全体合格率を示した
未学習の言い換え	1M・3 epochと15M・1 epochは15 / 30
指示の冒頭の変更	1M・3 epochは12→9、5M・1 epochは12→1（各14操作）

一種類のプロンプトでの精度に加え、表現を変えても性能が保たれるかを評価する必要がある。

この結果が示す範囲

- 訓練は24操作を最大3つ組み合わせるコード生成。追加評価では4操作まで調べた。
- 冒頭変更の比較は14操作。日本語能力全体の測定ではない。
- 学習は一つの乱数設定。繰り返し実験のばらつきは未確認。

今後は文型の偏りを減らした合成データで、表現への強さを再評価する。

Appendix D：Boku-nano 全体図

日本語の限定的な指示から solve(xs, k) を生成する小型Decoder-only Transformer。意味ASTを起点に、データ生成・学習・評価をすべて同じ意味仕様でつなぐ。

1 意味

24の基本操作
抽出 10：偶数・奇数・>k・≥k・<k・≤k・kの倍数・正・負・ゼロ
変換 8：+k・−k・×k・2倍・3倍・符号反転・絶対値・二乗
並べ替え 3／切り出し 3

→

意味AST　12,720仕様
24操作から1〜3操作を順序付きで選ぶ（重複なし）
24 ＋ 24×23 ＋ 24×23×22 ＝ 12,720
`{"sequence": [{"filter": ["even"]}, {"map": ["add_k"]}]}`
※日本語・コード・期待値・splitの共通の根拠

→

参照インタプリタ（期待値の計算）
意味ASTを入力リストに適用して、正しい出力を計算する
operator・itertools・heapq・deque・while で実装し、生成されるsolve関数の表現空間の外側に置く
→ 正解コードの検証（2）とhidden test採点（4）に使う
※参照実装と生成コードが同じ式を書いて誤りを共有しないため

↓ 意味ASTを分割　　↓ 期待値で検証（参照インタプリタ → 正解コード生成）

2 データ

意味ASTの分割
12,720 から 670 を組合せ汎化へ
残り 12,050 をハッシュ順で分割
　train　9,646（1操作24は全部）
　validation　1,202
　通常テスト　1,202（境界値も流用）
別枠：同一操作の反復 1,152
　　日本語言い換え 30指示

→

正解コード生成（自作テンプレート）
style_specでfor／内包表記・変数名・スライス表記などを変える
277,440候補 → 構文・Safe-AST・signature・入力非破壊・
41入力で参照インタプリタと出力一致、を全部通したものだけ

日本語指示生成
ルール：承認済み表現辞書（train 522件）を定型文へ連結
教師言い換え：Qwen3-4B-AWQ が意味と順序を保って言い換え
※Qwenは日本語だけを作り、コードや重みは提供しない

→

評価JSONL　108,590件
validation 24,040
正式5テスト 84,550問
hidden入力はコード作成時と別

訓練JSONL　192,900件
1操作 460／2操作 8,680／3操作 183,760
ルール文 96,510＋言い換え 96,390
1意味ASTあたり原則20件

⋯⋯

1レコードの中身
instruction_ja（日本語指示）
semantic_ast（意味AST）
reference_code（正解コード）
style_spec・code_style
split・tests・verification
SHA-256・seed・生成器version
教師モデル由来かどうかの来歴

↓ train の指示とコード（訓練JSONLから）

3 学習

BPEトークナイザ
ByteLevel BPE・語彙 2,048
trainの日本語指示と正解コードだけから学習
（評価データは入れない）
min_frequency 5・最大token長 24
unknown token 0件

→

学習系列とloss
`<|bos|><|task|> {instruction_ja}`
`<|code|> {reference_code}<|eos|>`
lossはコードとEOSだけ（指示は条件として読むだけ）
batch 512件・1エポック 377 step・AdamW・bf16

→

Decoder-only Transformer（ランダム初期値から学習）
RoPE・RMSNorm・SwiGLU・通常のMulti-Head Attention・最大系列長 256
　1M　d=128・3層・4 head
　5M　d=256・5層・4 head
　15M　d=384・8層・6 head
3つの規模を、それぞれ1エポック学習（RTX 5090）

↓ 学習済みモデル　　↓

4 評価

hidden test評価（正式5テスト 84,550問）
greedy生成 → 構文 → Safe-AST → signature → 隔離実行
→ hidden入力すべてで参照インタプリタと同じ出力なら合格
文字列一致ではなく、実行結果で判定する
　通常 24,040　　組合せ汎化 13,400　　日本語言い換え 30
　同一操作の反復 23,040　　境界値 24,040
※attention診断の24問（1操作）はこれとは別の解析用テスト

→

主な結果（1エポック・正式5テスト合計）
15M　99.56%（84,180 / 84,550）
5M　96.46%（81,557 / 84,550）
1M　1.74%（1,467 / 84,550）
※弱点：未学習の言い換え（15Mでも30問中15問）

内部解析
Q・K・Vを取り出しattentionを計算
head別のattention配分・Value norm・
BOS sink・attention rollout
head／位置ablation：NLL差（nat）と
再生成コードの合格数で因果を確認

5 デモ

自由な日本語
例：全部プラスにして大きい順に並べて

→

Qwen3-0.6B（ブラウザ）
自由文をCNLへ翻訳するだけ
追加学習なし・コードは書かない

→

JavaScript検査
24操作・最大3操作・
定型文法に合うか確認

→

Boku1-nano ONNX
学習済みモデルを選んで
ブラウザ内でgreedy生成

→

Pythonコード
`def solve(xs, k) -> list[int]`

自作ルール・決定的な処理　Qwenを使う処理（日本語のみ）　データ・出力　学習・モデル　評価・解析　↓ 参照インタプリタによる検証